Нелинейные методы классификации

Паточенко Евгений

НИУ ВШЭ

План занятия

- Наивный байесовский классификатор
- kNN
- Kernel SVM

Наивный байесовский классификатор

Определение

Наивный байесовский классификатор (Naïve Bayes) — алгоритм обучения с учителем, основанный на применении теоремы Байеса с «наивным» предположением об условной независимости между каждой парой признаков при заданном значении переменной класса

Наивный байесовский классификатор

Теорема Байеса

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}, \text{где}$$

$P(A)$ — априорная вероятность события A до наблюдения события B

$P(A|B)$ — апостериорная вероятность события A после наблюдения события B

$P(B|A)$ — вероятность наступления события B при наступлении события A

$P(B)$ — полная вероятность события B

Наивный байесовский классификатор

Описание метода

Для задачи классификации мы можем воспользоваться универсальностью теоремы и под событием A иметь в виду метку класса Y некоторого объекта, под событием B — признаки X этого объекта:

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

Наивный байесовский классификатор

Описание метода

Для задачи классификации мы можем воспользоваться универсальностью теоремы и под событием A иметь в виду метку класса Y некоторого объекта, под событием B — признаки X этого объекта:

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

или

$$P\left(y_j \middle| x_1, x_2, \dots x_d\right) = \frac{P\left(x_1, x_2, \dots x_d \middle| y_j\right) \cdot P(y_j)}{P(x_1, x_2, \dots x_d)}$$

Наивный байесовский классификатор

Описание метода

Для задачи классификации мы можем воспользоваться универсальностью теоремы и под событием A иметь в виду метку класса Y некоторого объекта, под событием B — признаки X этого объекта:

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

или $$P\left(y_j \middle| x_1, x_2, \dots x_d\right) = \frac{P\left(x_1, x_2, \dots x_d \middle| y_j\right) \cdot P(y_j)}{P(x_1, x_2, \dots x_d)}$$

Тогда в качестве ответа выдаем класс с наибольшей вероятностью:

$$\boldsymbol{y = argmax_{i=1,\dots K} \frac{P\left(x_1, x_2, \dots x_d \middle| y_j\right) \cdot P(y_j)}{P(x_1, x_2, \dots x_d)}}$$

Наивный байесовский классификатор

Описание метода

Так как в формуле предсказания нас интересует не точное значение вероятности, а только какой класс имеет наибольшее значение вероятности, то знаменатель можно не вычислять:

$$y = argmax_{i=1,\dots K} P\left(x_1, x_2, \dots x_d \middle| y_j\right) \cdot P(y_j)$$

Наивный байесовский классификатор

Описание метода

Так как в формуле предсказания нас интересует не точное значение вероятности, а только какой класс имеет наибольшее значение вероятности, то знаменатель можно не вычислять:

$$y = argmax_{i=1,\dots K} P\left(x_1, x_2, \dots x_d \middle| y_j\right) \cdot P(y_j)$$

Считаем, что события независимы. Значит:

$$\boldsymbol{P\left(x_1, x_2, \dots x_d \middle| y_j\right) = P\left(x_1 \middle| y_j\right) \cdot P\left(x_2 \middle| y_j\right) \cdot \dots P\left(x_d \middle| y_j\right)}$$

Наивный байесовский классификатор

Сглаживание Лапласа

- **Проблема.** Слово ни разу не встретилось в спаме: P(слово | спам) = 0, и все произведение обращается в ноль, какие бы другие слова ни были в письме
- **Решение.** Прибавить к каждому счетчику малую константу α (в sklearn параметр alpha, по умолчанию 1):

$$P(x_i \mid y_j) = \frac{n_{ij} + \alpha}{n_j + \alpha \cdot V}$$

n_{ij} — сколько раз признак i встретился в классе j; n_j — число всех событий в классе; V — размер словаря

Сообщение	α → 0	α = 1
бесплатно приз выигрыш	1,000	0,974
бесплатно завтра	0,000	0,435
бесплатно встреча завтра	0,000	0,206

Наивный байесовский классификатор

Гауссов наивный байесовский классификатор

Часто используется гауссов наивный байес

(приближает численные признаки нормальным распределением):

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

Параметры μ_y и σ_y оцениваются с использованием метода максимального правдоподобия

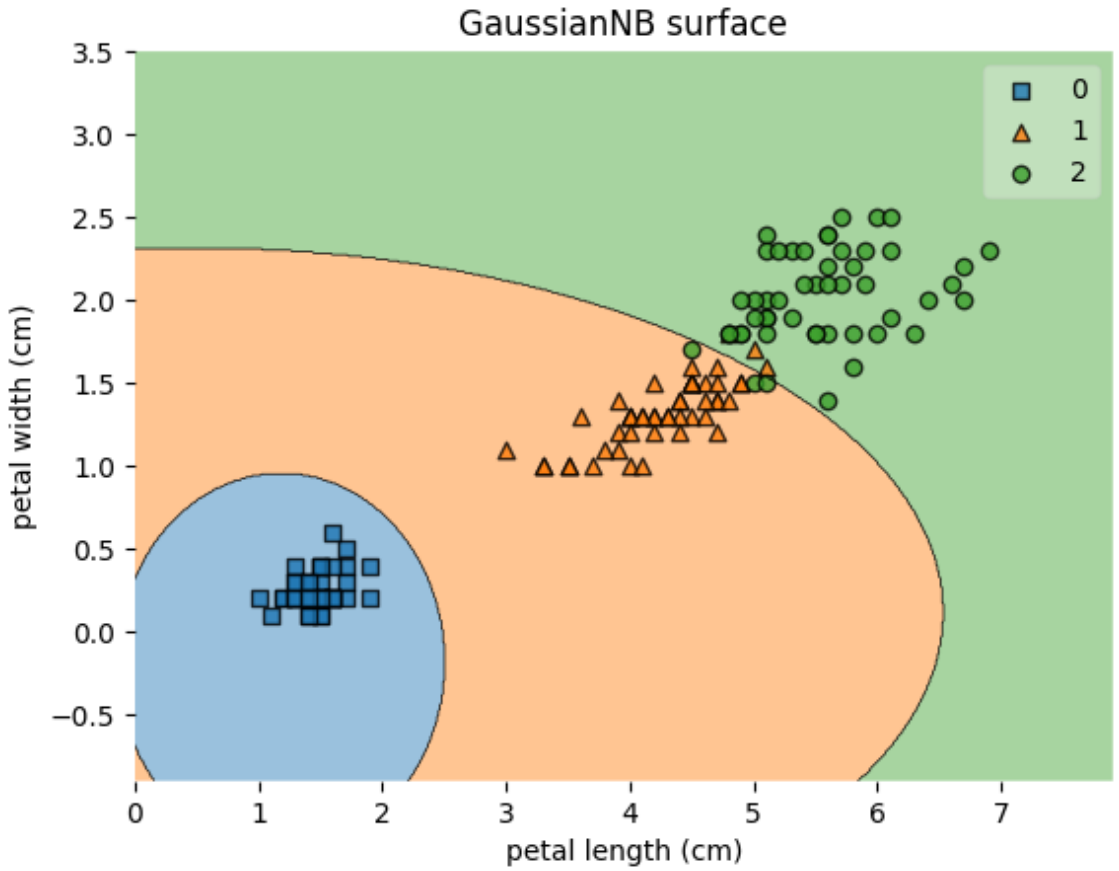
Наивный байесовский классификатор

Примеры других реализаций

- многочленный наивный байес — для полиномиально распределенных данных и классификации текста (частая задача — определение спама)
- наивный байесовский алгоритм дополнения — для несбалансированных данных
- наивный байес Бернулли — для данных с многомерным распределением Бернулли
- категориальный наивный байес — для категориальных данных

Документация scikit-learn

Наивный байесовский классификатор



Пример результата работы алгоритма по классификации сортов ириса

Наивный байесовский классификатор

Применимость

Преимущества:

Наивный байесовский классификатор

Применимость

Преимущества:

- Простота реализации

Наивный байесовский классификатор

Применимость

Преимущества:

- Простота реализации
- Низкие вычислительные затраты

Наивный байесовский классификатор

Применимость

Преимущества:

- Простота реализации
- Низкие вычислительные затраты
- Если признаки действительно независимы, метод оптимален

Наивный байесовский классификатор

Применимость

Преимущества:

- Простота реализации
- Низкие вычислительные затраты
- Если признаки действительно независимы, метод оптимален

Недостатки:

Наивный байесовский классификатор

Применимость

Преимущества:

- Простота реализации
- Низкие вычислительные затраты
- Если признаки действительно независимы, метод оптимален

Недостатки:

- В большинстве реальных задач признаки не полностью независимы, поэтому алгоритм показывает низкое качество

Метод ближайших соседей (kNN)

Определение

Метод ближайших соседей (k-nearest neighbors, kNN) — метрический алгоритм классификации или регрессии, основанный на *гипотезе компактности*: похожие объекты лежат в одном классе (или имеют схожее значение целевой переменной в задачах регрессии)

Алгоритм не строит модель на этапе обучения, а просто запоминает всю обучающую выборку (lazy learning). Все вычисления происходят только на этапе прогнозирования

Метод ближайших соседей (kNN)

Логика работы



Метод ближайших соседей (kNN)

Логика работы

Чтобы классифицировать новый объект, нужно:

- вычислить расстояние до каждого из объектов обучающей выборки
- выбрать k объектов обучающей выборки, расстояние до которых минимально

Класс нового объекта – это класс, наиболее часто встречающийся среди k ближайших соседей

Метод ближайших соседей (kNN)

Логика работы

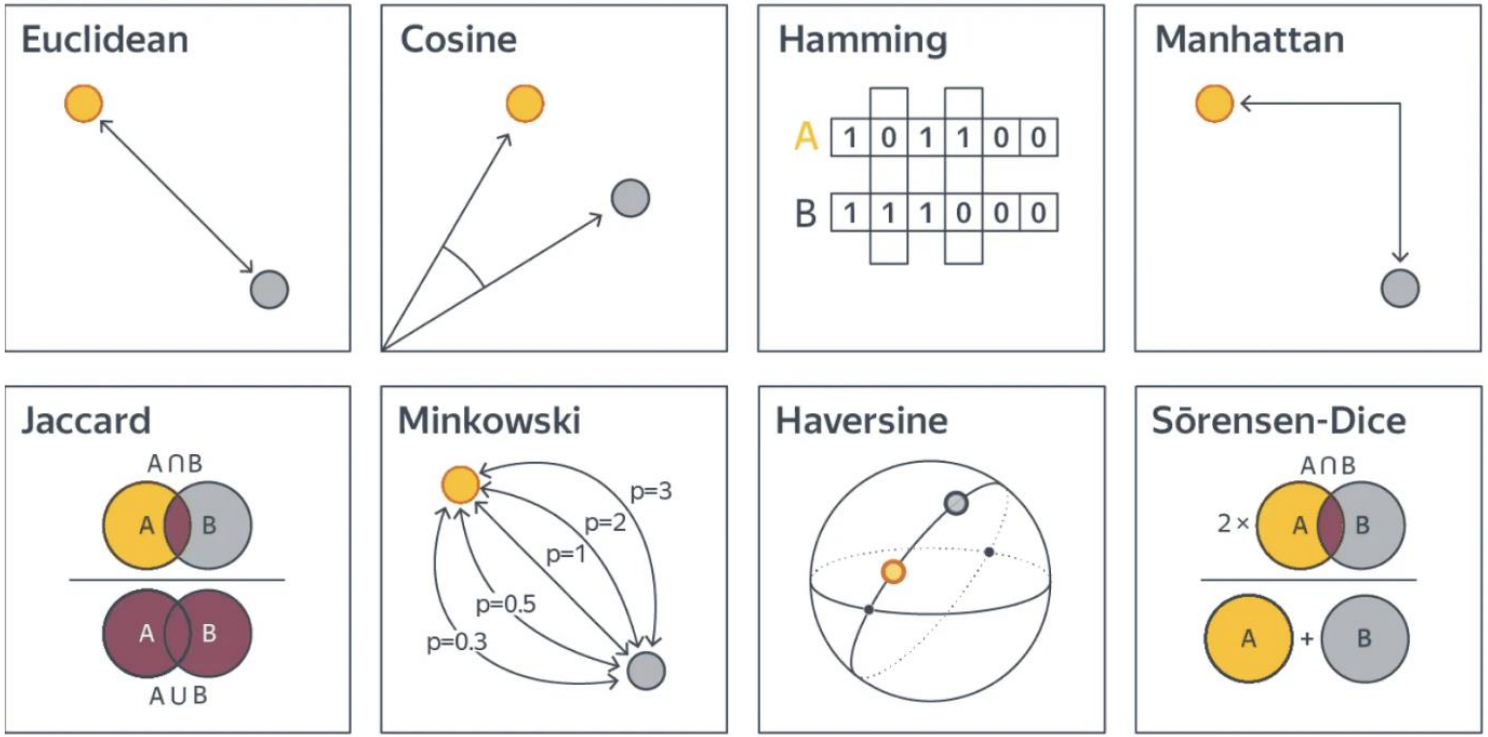
Чтобы классифицировать новый объект, нужно:

- **вычислить расстояние до каждого из объектов обучающей выборки**
- выбрать k объектов обучающей выборки, расстояние до которых минимально

Класс нового объекта – это класс, наиболее часто встречающийся среди k ближайших соседей

Метод ближайших соседей (kNN)

Метрики



Метод ближайших соседей (kNN)

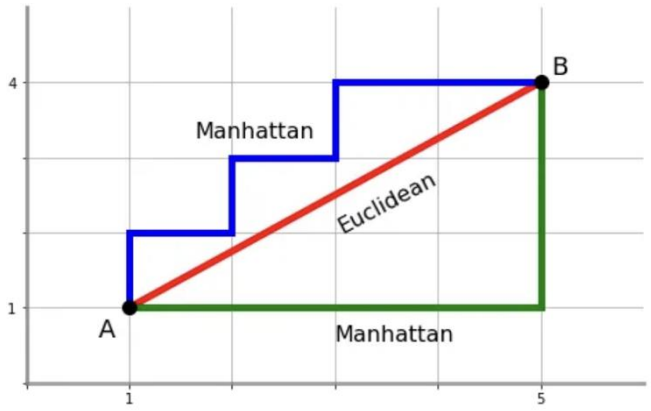
Метрики

- Наиболее частый выбор, если значения признаков непрерывные, – **евклидово расстояние**: $\sqrt{\sum_{i=1}^{d}(a_i - b_i)^2}$

Метод ближайших соседей (kNN)

Метрики

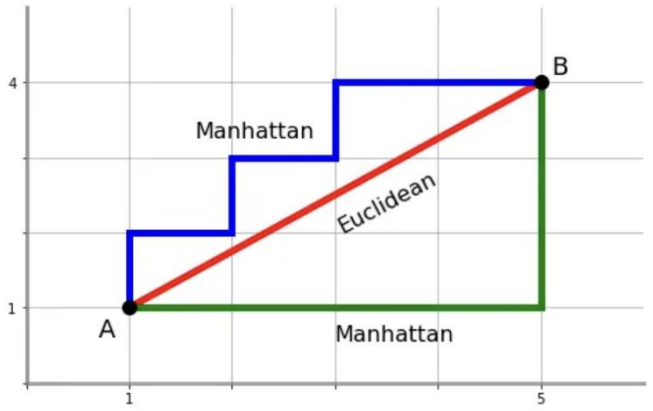
- Наиболее частый выбор, если значения признаков непрерывные, – **евклидово расстояние**: $\sqrt{\sum_{i=1}^{d}(a_i - b_i)^2}$
- Лучше работает на данных с выбросами — **манхэттенская метрика:** $\sum_i |a_i - b_i|$

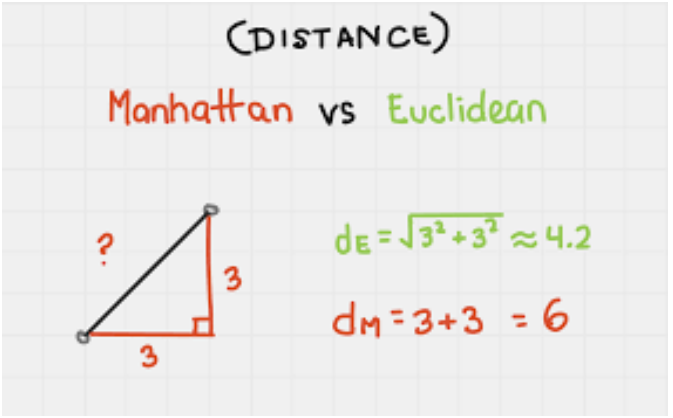


Метод ближайших соседей (kNN)

Метрики

- Наиболее частый выбор, если значения признаков непрерывные, – **евклидово расстояние**: $\sqrt{\sum_{i=1}^{d}(a_i - b_i)^2}$
- Лучше работает на данных с выбросами — **манхэттенская метрика:** $\sum_i |a_i - b_i|$





Метод ближайших соседей (kNN)

Метрики

- Обобщение евклидова расстояния и манхэттенского — **метрика Минковского:**

$$\sqrt[p]{\sum_i |a_i - b_i|^p}$$

Метод ближайших соседей (kNN)

Метрики

- Обобщение евклидова расстояния и манхэттенского — **метрика Минковского:**

  $\sqrt[p]{\sum_i |a_i - b_i|^p}$

- Частный случай метрики Минковского для категориальных признаков — **расстояние Хэмминга:** $\sum_{k=1}^{p} |a_{ik} - b_{jk}|$

Метод ближайших соседей (kNN)

Метрики

- Обобщение евклидова расстояния и манхэттенского — **метрика Минковского:**

  $\sqrt[p]{\sum_i |a_i - b_i|^p}$

- Частный случай метрики Минковского для категориальных признаков — **расстояние Хэмминга:** $\sum_{k=1}^{p} |a_{ik} - b_{jk}|$

- Для текстовых данных — **косинусное расстояние**: $1 - \cos\theta = 1 - \frac{a \cdot b}{\|a\|\|b\|}$

Метод ближайших соседей (kNN)

Логика работы

Чтобы классифицировать новый объект, нужно:

- вычислить расстояние до каждого из объектов обучающей выборки
- **выбрать k объектов обучающей выборки, расстояние до которых минимально**

Класс нового объекта – это класс, наиболее часто встречающийся среди k ближайших соседей

Метод ближайших соседей (kNN)

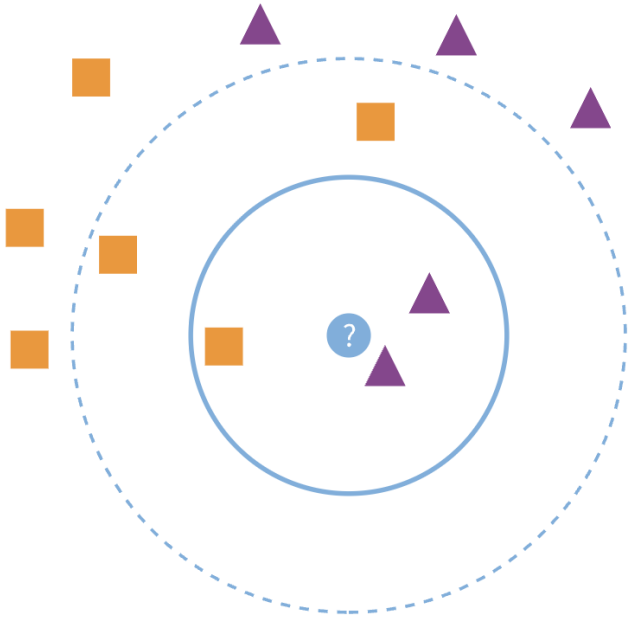
Гиперпараметр k

Мы сами выбираем значение гиперпараметра k — числа ближайших соседей, на которые мы смотрим для определения класса

Метод ближайших соседей (kNN)

Гиперпараметр k

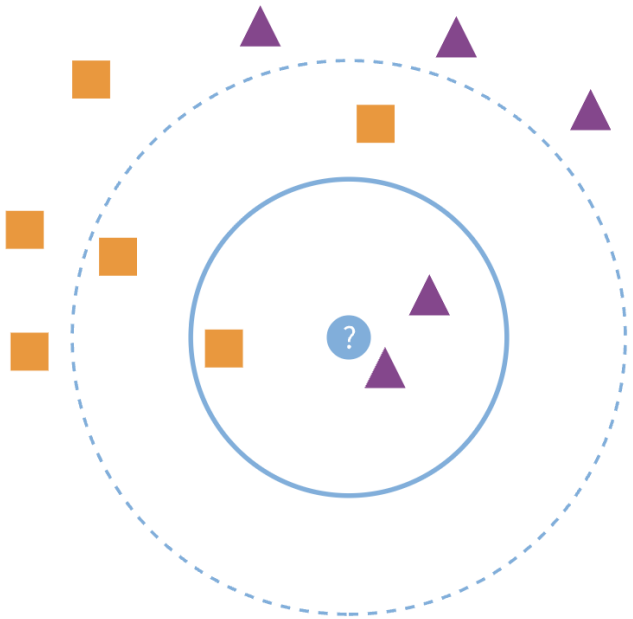
В зависимости от выбранного значения k результаты классификации будут меняться



Метод ближайших соседей (kNN)

Гиперпараметр k

В зависимости от выбранного значения k результаты классификации будут меняться

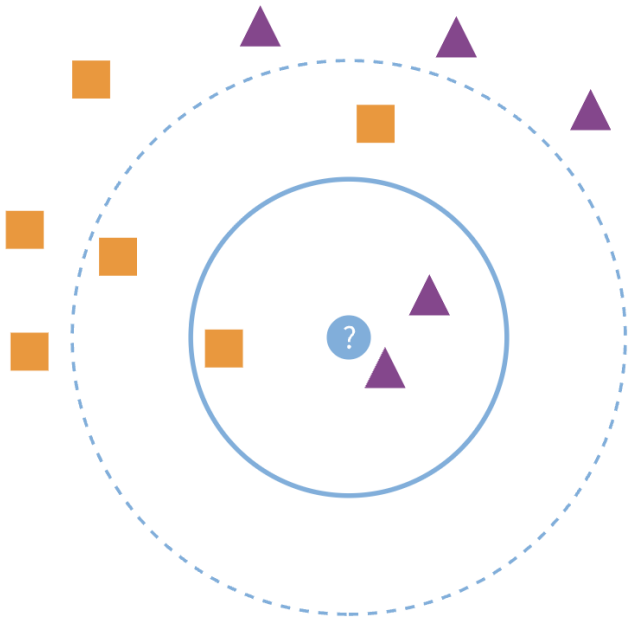


К какому классу будет отнесен объект при $k = 3$?

Метод ближайших соседей (kNN)

Гиперпараметр k

В зависимости от выбранного значения k результаты классификации будут меняться



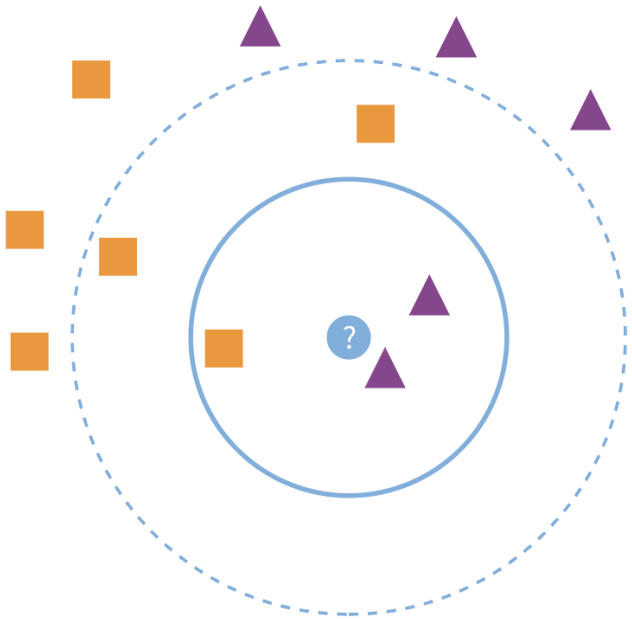
К какому классу будет отнесен объект при $k = 3$?

Треугольник

Метод ближайших соседей (kNN)

Гиперпараметр k

В зависимости от выбранного значения k результаты классификации будут меняться



К какому классу будет отнесен объект при $k = 3$?

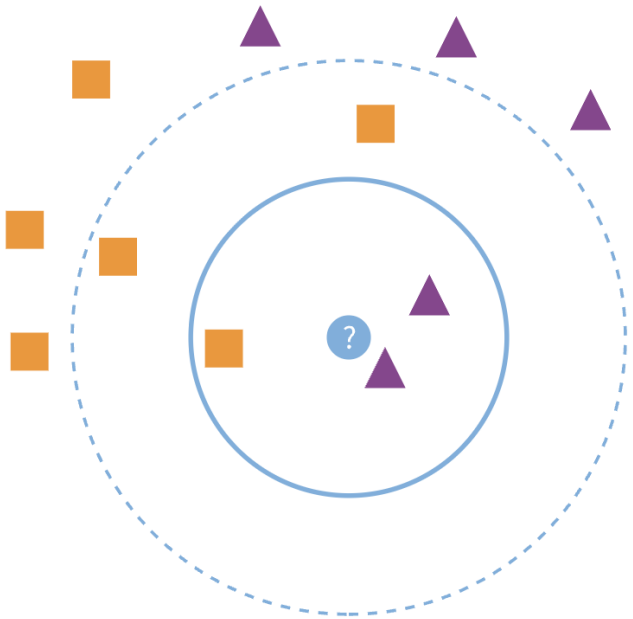
Треугольник

К какому классу будет отнесен объект при $k = 5$?

Метод ближайших соседей (kNN)

Гиперпараметр k

В зависимости от выбранного значения k результаты классификации будут меняться



К какому классу будет отнесен объект при $k = 3$?

Треугольник

К какому классу будет отнесен объект при $k = 5$?

Квадрат

Метод ближайших соседей (kNN)

Гиперпараметр k

Как будет работать алгоритм при $k = 1$?

Метод ближайших соседей (kNN)

Гиперпараметр k

Как будет работать алгоритм при $k = 1$?

Слишком малое k создает эффект переобучения. При $k = 1$ алгоритм будет присваивать любому новому наблюдению метку класса ближайшего объекта

Метод ближайших соседей (kNN)

Гиперпараметр k

Как будет работать алгоритм при $k = 1$?

Слишком малое k создает эффект переобучения. При $k = 1$ алгоритм будет присваивать любому новому наблюдению метку класса ближайшего объекта

При больших значениях k уменьшается влияние шумов в данных, но снижается выраженность границ классов и классификация становится слишком грубой

Метод ближайших соседей (kNN)

Гиперпараметр k

Как будет работать алгоритм при $k = 1$?

Слишком малое k создает эффект переобучения. При $k = 1$ алгоритм будет присваивать любому новому наблюдению метку класса ближайшего объекта

При больших значениях k уменьшается влияние шумов в данных, но снижается выраженность границ классов и классификация становится слишком грубой

Выбор значения k сводится к компромиссу между точностью и обобщающей способностью модели

Метод ближайших соседей (kNN)

Формализация

Пусть k – количество соседей. Для каждого объекта u возьмем k ближайших к нему объектов из тренировочной выборки:

$$x_{(1;u)}, x_{(2;u)}, \dots, x_{(k;u)}$$

Тогда класс объекта u определяется следующим образом:

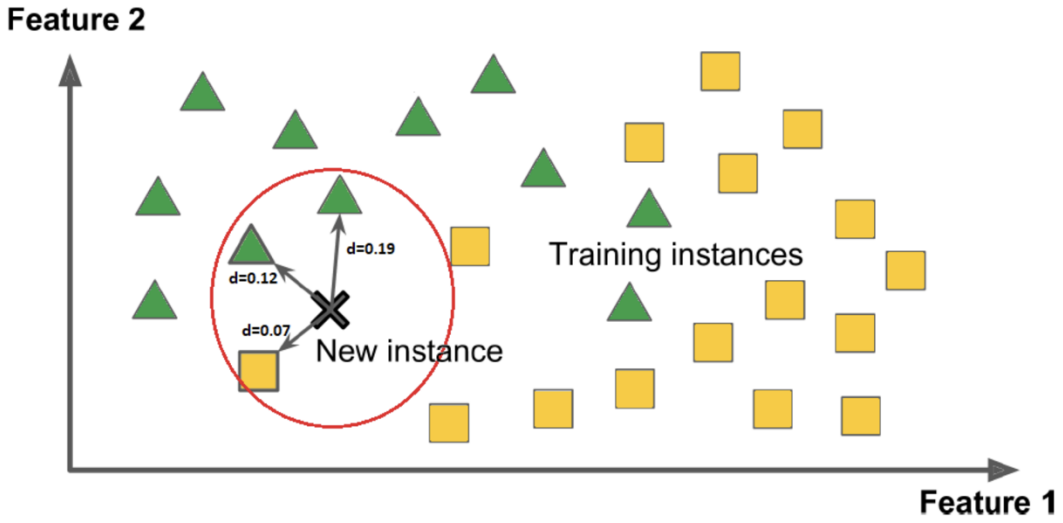
$$argmax_{y \in Y} \sum_{i=1}^{k} I\left[y\left(x_{(i;u)}\right) = y\right]$$

Метод ближайших соседей (kNN)

Взвешенный kNN

Чем ближе объекты в пространстве признаков, тем выше их сходство. Для учета близости при предсказании класса нового объекта используем веса: ближайшие соседи получают больший вес:

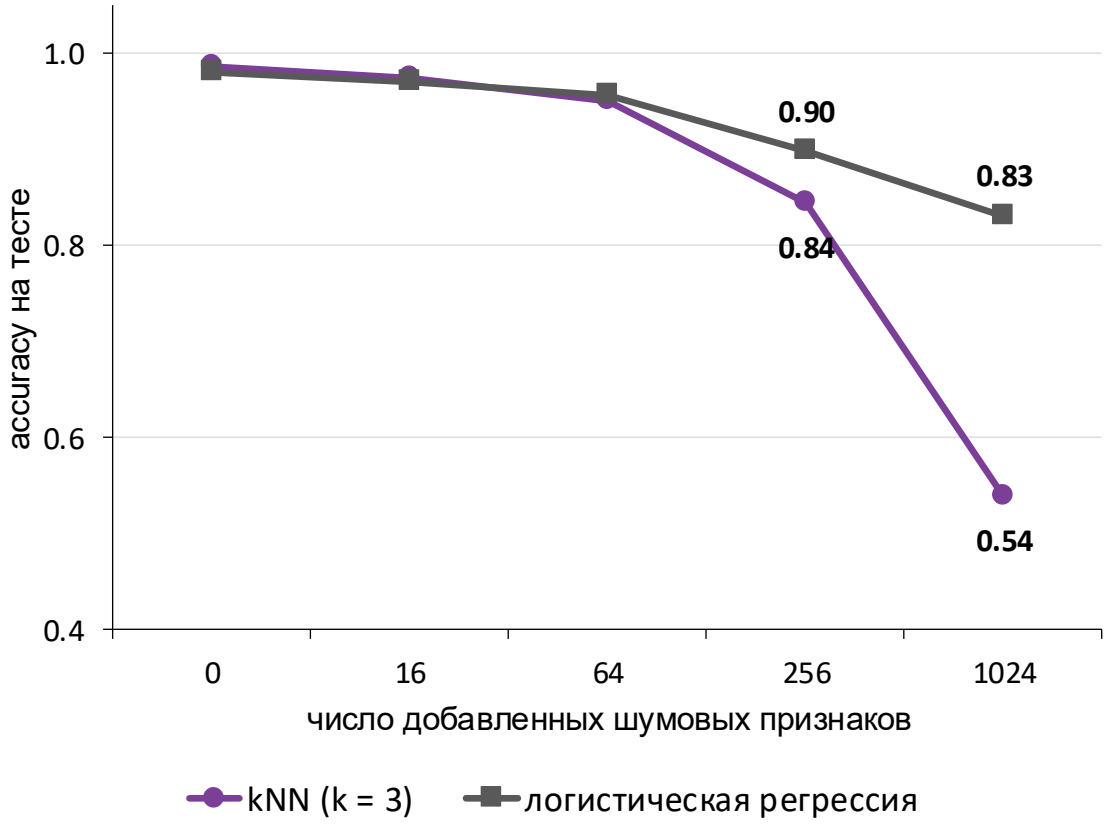
$$argmax_{y \in Y} \sum_{i=1}^{k} \frac{1}{p(u, x_{(i;u)})} I[y(x_{(i;u)}) = y]$$



Метод ближайших соседей (kNN)

Проклятие размерности

- Расстояние складывается из вкладов всех признаков. Шумовой признак вносит случайный вклад, а у kNN нет весов, чтобы его «отключить»
- Цифры (64 пикселя) и добавленные шумовые признаки: при 1024 лишних признаках точность kNN падает с 0,99 до 0,54, у логистической регрессии только до 0,83
- В высокой размерности расстояния «слипаются»: самая далекая точка почти так же близка, как самая близкая, и «ближайший сосед» теряет смысл
- **Что делать:** отбирать признаки, снижать размерность (PCA), масштабировать



Рукописные цифры sklearn, шум равномерный от 0 до 16

Метод ближайших соседей (kNN)

Применимость

Преимущества:

- Простота реализации
- Не требует обучения (lazy learning)
- Высокая интерпретируемость

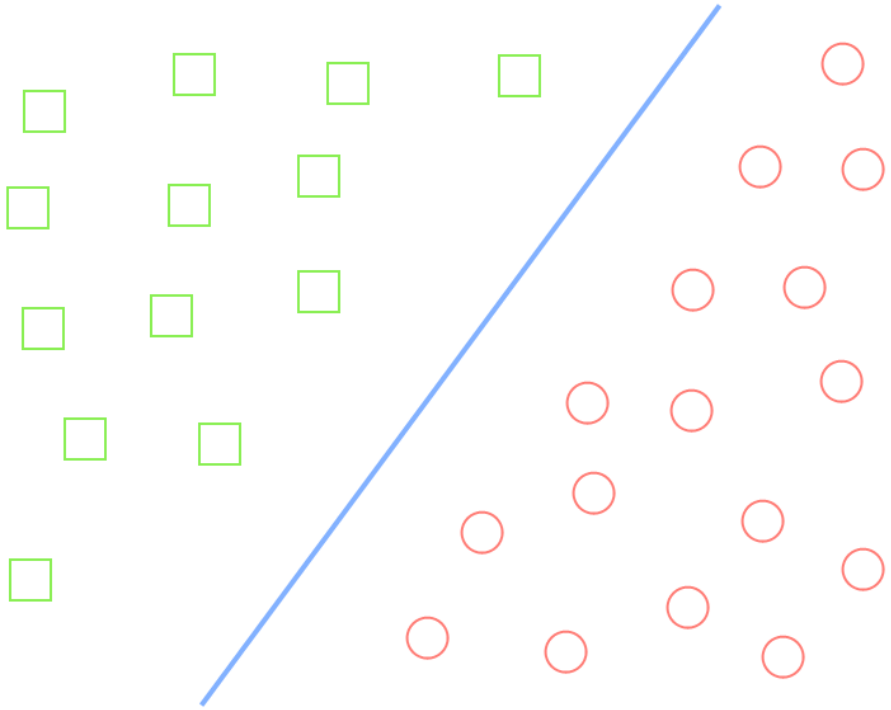
Недостатки:

- Дорогой по вычислениям на этапе предсказания (нужно считать расстояния до всех объектов)
- Чувствителен к масштабу признаков (требует нормализации)

SVM (напоминание)

Линейно разделимая выборка

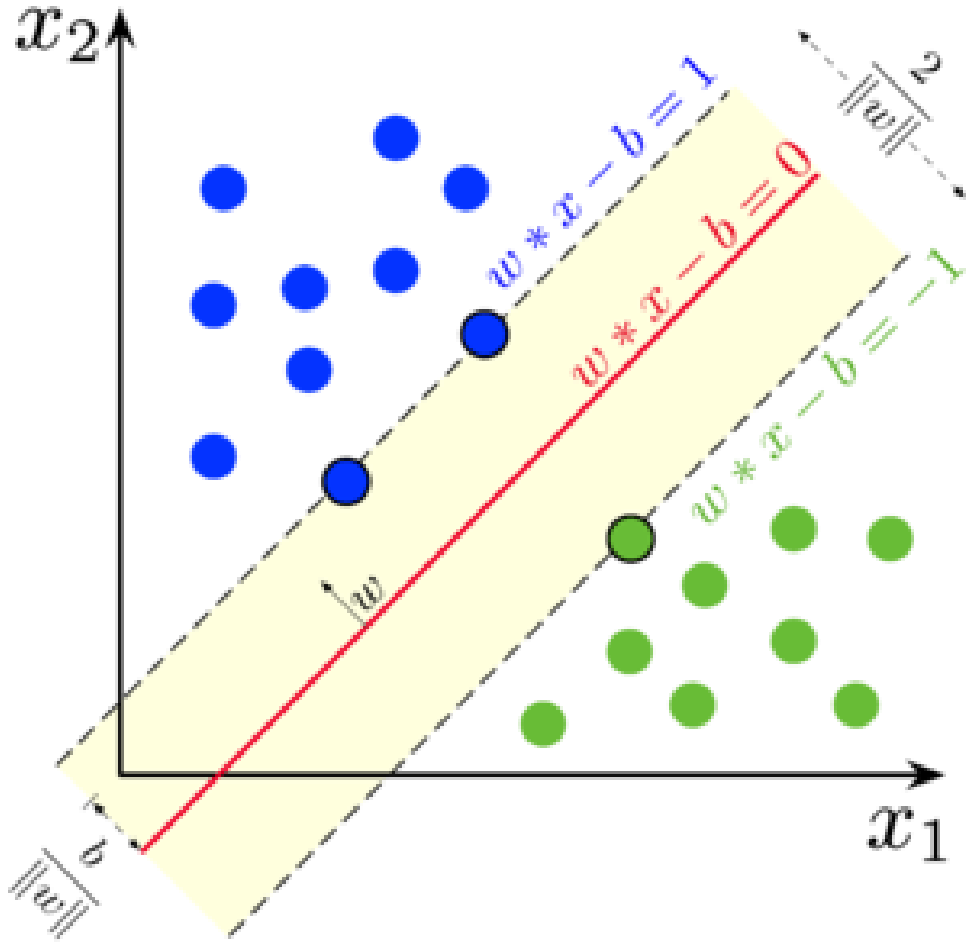
Выборка линейно разделима, если существует такой вектор параметров β, что соответствующий классификатор $f(x)$ не допускает ошибок на этой выборке



SVM (напоминание)

SVM (Support Vector Machine) для линейно разделимого случая

Цель SVM — максимизировать ширину разделяющей полосы, то есть найти наилучшую разделяющую гиперплоскость, которая максимально отдалена от ближайших точек каждого класса (**опорных векторов**)



SVM (напоминание)

Формулы

- Решающая функция (w — вектор весов, b — свободный член):

$$f(x) = \mathrm{sign}(\langle w, x\rangle + b), \quad \text{граница: } \langle w, x\rangle + b = 0$$

- Ширина полосы равна 2 / ‖w‖, SVM ее максимизирует. «Мягкий» SVM разрешает объектам нарушать полосу и штрафует за это:

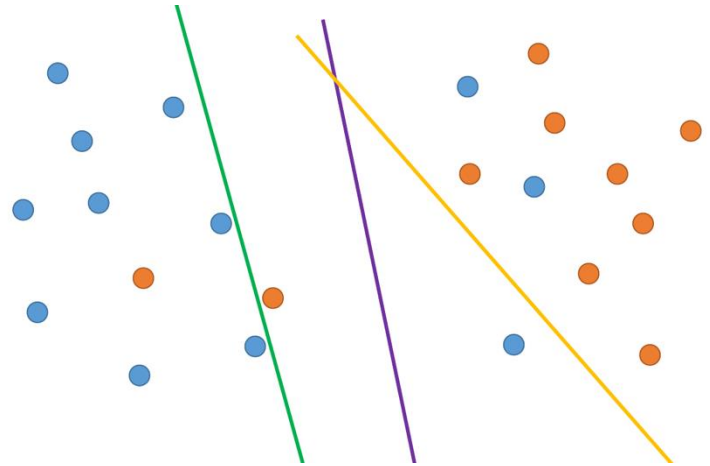
$$\tfrac{1}{2}\|w\|^2 + C \cdot \Sigma_i \max(0, 1 - y_i(\langle w, x_i\rangle + b)) \to \min$$

- C — цена нарушения: большое C дает узкую полосу и мало нарушений, малое C — широкую полосу и много нарушений

Kernel SVM

Линейно неразделимая выборка

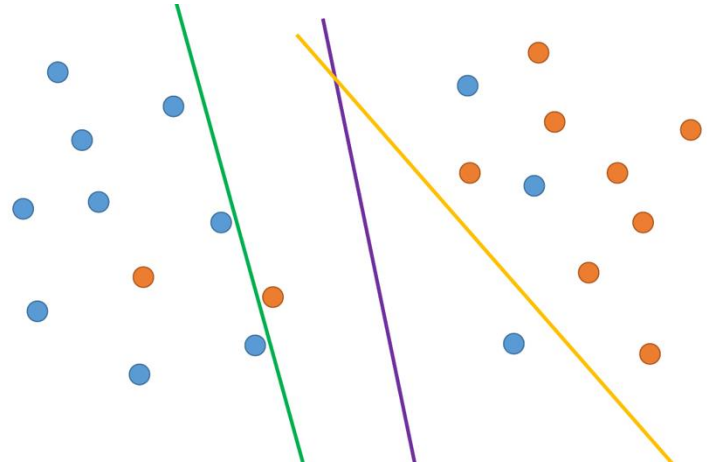
В реальных задачах в исходном пространстве признаков данные гораздо чаще *линейно неразделимы*



Kernel SVM

Линейно неразделимая выборка

В реальных задачах в исходном пространстве признаков данные гораздо чаще *линейно неразделимы*

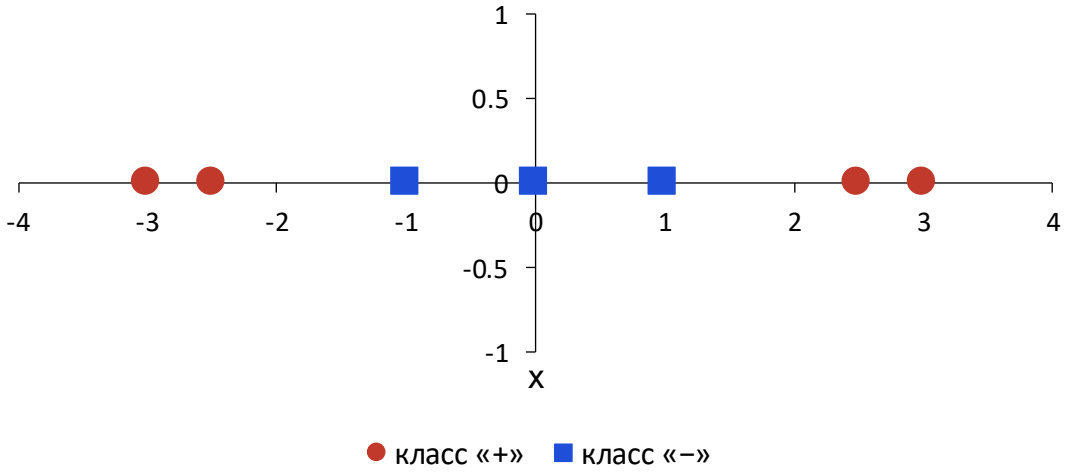


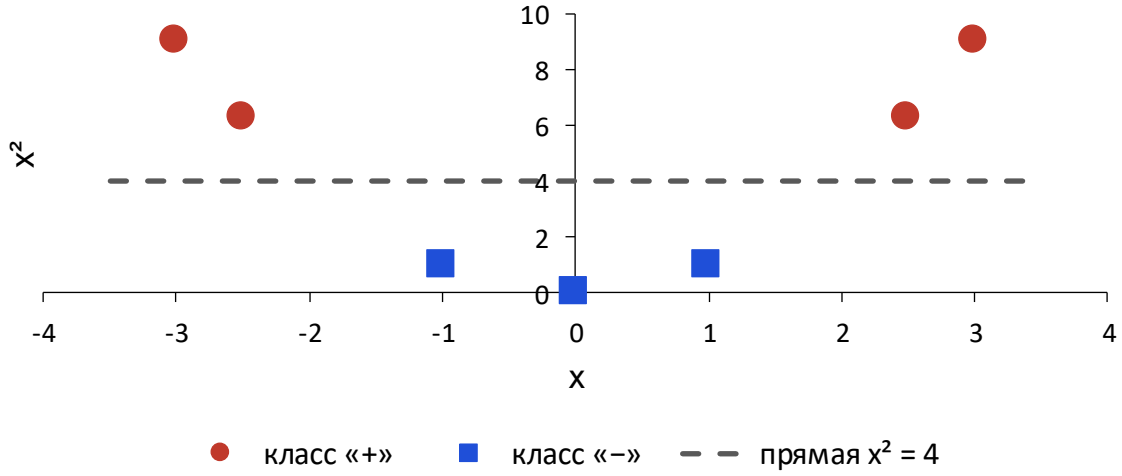
Возможное решение — добавить полиномиальные признаки

Kernel SVM

Идея: добавить признак

Один признак x: одной точкой классы не разделить («минусы» посередине). Добавим признак x^2: в плоскости (x, x^2) их разделяет прямая $x^2 = 4$, а в исходных координатах это две точки, $x = -2$ и $x = 2$



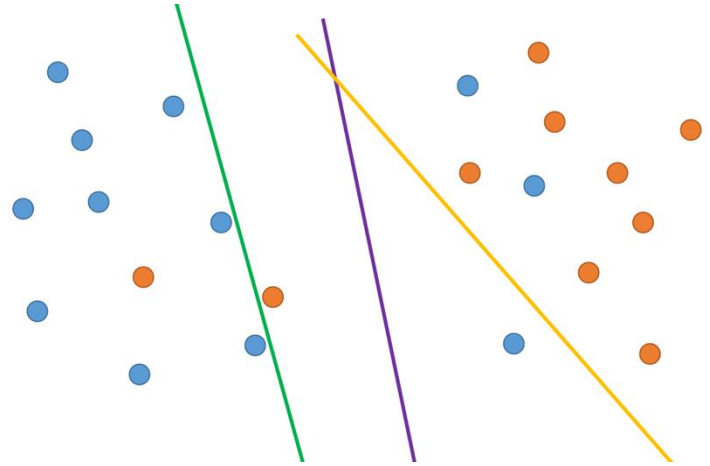


Шум лечится «мягким» зазором (параметр C), а искривленную границу — новыми признаками или ядром

Kernel SVM

Линейно неразделимая выборка

В реальных задачах в исходном пространстве признаков данные гораздо чаще *линейно неразделимы*



Возможное решение — добавить полиномиальные признаки

Проблема: число признаков растет очень быстро. Для 100 исходных признаков и степени 3 получается 171 700 новых признаков, для 784 пикселей и степени 2 — 307 720. Вычислительные расходы растут вместе с ними

Kernel SVM

Kernel trick

- SVM использует объекты только через скалярные произведения. Решающая функция (сумма по опорным объектам, α — веса, найденные при обучении):

$$f(x) = \mathrm{sign}(\Sigma_{i \in SV}\, \alpha_i\, y_i\, \langle x_i, x \rangle + b)$$

- Если признаки заменить на ϕ(x), понадобятся только произведения ⟨ϕ(a), ϕ(b)⟩, а сами векторы ϕ(a) не нужны. Ядро — функция, которая считает такое произведение напрямую, не строя ϕ:

$$K(a, b) = \varphi(a)^T\, \varphi(b)$$

- Поэтому в решающей функции $\langle x_i, x \rangle$ просто заменяют на $K(x_i, x)$, остальной алгоритм не меняется. Это дает сложные границы без взрывного роста числа признаков

Kernel SVM

Пример ядра: степень 2

- Два признака, $a = (a_1, a_2)$. Добавим признаки степени 2:

$$\varphi(a) = (a_1^2,\ \sqrt{2}\cdot a_1 a_2,\ a_2^2)$$

- Скалярное произведение в новом пространстве оказывается квадратом обычного:

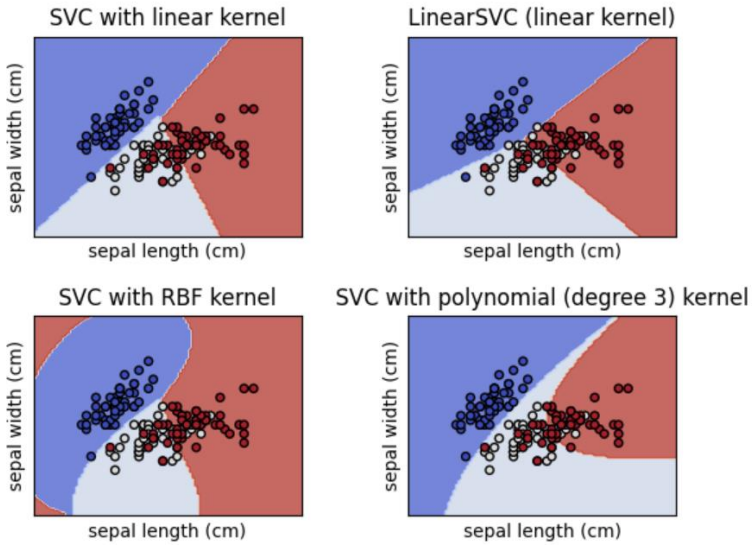
$$\varphi(a)^T\varphi(b) = a_1^2 b_1^2 + 2a_1 a_2 b_1 b_2 + a_2^2 b_2^2 = (a_1 b_1 + a_2 b_2)^2 = (a^T b)^2$$

- Проверка: $a = (1, 2)$, $b = (3, 1)$. Тогда $\phi(a) = (1; 2\sqrt{2}; 4)$, $\phi(b) = (9; 3\sqrt{2}; 1)$, $\phi(a)^T\phi(b) = 9 + 12 + 4 = 25 = (1\cdot 3 + 2\cdot 1)^2$
- Для 100 признаков и степени 3 это 171 700 признаков в явном виде против сотни умножений в ядре. У RBF-ядра пространство признаков бесконечномерно, а ядро считается одной формулой

Kernel SVM

Ядро

Ядро можно читать и как меру сходства двух объектов: чем ближе объекты, тем больше K(a, b). Решающая функция SVM — взвешенное голосование опорных объектов по их сходству с новым объектом



Ирисы, два признака. Сверху линейные ядра: SVC(kernel="linear") и LinearSVC. Это разные реализации (другая функция потерь), поэтому границы немного различаются. Снизу: RBF и полином степени 3

Kernel SVM

Примеры ядер

- **Линейное:** $K(a, b) = a^T b$ — обычный линейный SVM
- **Полиномиальное:** $K(a, b) = (\gamma\, a^T b + r)^d$ — улавливает более сложные зависимости
  - d — степень (не число признаков): чем она больше, тем более изогнута граница и выше риск переобучения
  - γ — множитель при скалярном произведении, r (в sklearn coef0) — сдвиг: при r = 0 остаются только члены степени ровно d, при r > 0 и младших степеней

Kernel SVM

Примеры ядер: RBF (гауссово)

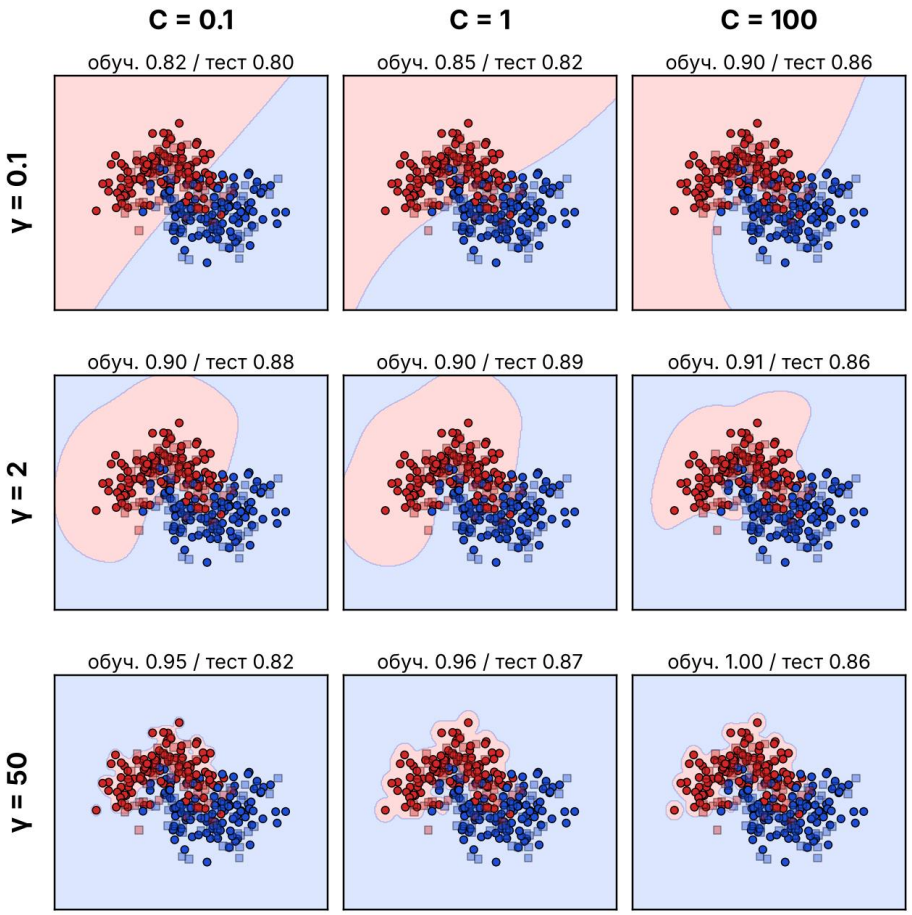
$$K(a, b) = \exp(-\gamma \|a - b\|^2)$$

- Значение зависит только от расстояния между объектами: у совпадающих объектов оно равно 1, с ростом расстояния быстро убывает к 0. Каждый опорный объект создает вокруг себя «холмик» влияния, решающая функция — сумма холмиков с весами
- Самое универсальное ядро: подходит для данных сложной нелинейной формы. Но и самое гибкое: при большом γ и большом C оно легко переобучается (следующий слайд)
- Ядро зависит от евклидова расстояния, поэтому признаки нужно стандартизовать

Kernel SVM

RBF: параметры γ и C

- **γ** — ширина «холмика» влияния каждого объекта. Большое γ: узкие холмики, граница обтекает отдельные точки (как kNN с малым k). Малое γ: гладкая, почти линейная граница
- **C** — цена нарушения зазора. Большое C: модель любой ценой не ошибается на обучении. Малое C: широкий зазор и много нарушений
- Слишком большие γ и C дают переобучение: при γ = 50, C = 100 точность на обучении 1,00, на тесте 0,86, а лучший тест 0,89 у γ = 2, C = 1
- Параметры подбирают вместе по логарифмической сетке с кросс-валидацией, признаки перед этим стандартизуют



Две луны с шумом: 210 обучающих (круги) и 90 тестовых (светлые квадраты)

Kernel SVM

Как выбирать ядро

- Порядок: сначала линейное (быстро, при большом числе признаков часто достаточно), затем RBF. Полиномиальное (степень 2–3) — если важны попарные взаимодействия признаков
- Сигмоидальное, $K(a, b) = \tanh(\gamma\, a^T b + r)$, почти не используют: оно не при всех параметрах является корректным ядром
- Всегда: стандартизовать признаки, подбирать C и γ кросс-валидацией на одних и тех же разбиениях
- Цена: время обучения растет быстрее линейного по числу объектов (комфортно до десятков тысяч), вероятностей «из коробки» нет
- Корректное ядро: матрица $K(x_i, x_j)$ симметрична и неотрицательно определена